

Assignment:- 2

Name: Kaushal Kumar Raj

Branch: B-Tech CSE with AI & ML

PRN: 240205241023

Subject: DBMS

Semester: III

① Define functional dependency with an example?

→ A functional dependency means that one attribute (or set of attributes) uniquely determines another attribute in a relation.

Example: In a student table, Roll No $\rightarrow$ Name. Each Roll No has only one Name, so Name depends on Roll No.

② What is a transitive dependency?

→ This occurs when a non-prime attribute depends indirectly on the primary key through another attribute.

Example:

Roll No $\rightarrow$ DeptID and DeptID $\rightarrow$ DeptName. Here, DeptName depends on Roll No through DeptID.

③ Explain the concept of partial dependency?

→ A partial dependency happens when a non-key attribute depends only on part of a composite primary key, not the whole key.

Example: In (Roll No, courseID) $\rightarrow$ marks, if Roll No $\rightarrow$ studentName, then,

studentName depends only on Roll No, not on the Roll No.

④ Define normalization. why is it needed?

- Normalization is the process of structuring a database to minimize redundancy and dependency problems. It breaks big tables into smaller, well-structured ones.
- It prevents anomalies (insertion, update, deletion), reduces duplicate data, and improves data consistency.

⑤ Differentiate between 1NF and 2NF.

- 1NF:
A table is in 1NF if all attributes have atomic values like no arrays, no repeating groups.

2NF:

A table is in 2NF if it is already in 1NF and has no partial dependencies (every non-key attribute depends on the full primary key).

⑥ What is 3NF? Give an example.

- A relation is in 3NF if it is in 2NF and has no transitive dependencies - non-key attributes must depend only on candidate keys, not on other non-key attributes.

Example:

If DeptID → DeptName, move DeptName into a separate Dept table to eliminate transitive dependency.

⑦ Define BCNF and explain how is it different from 3NF.

→ BCNF (Boyce-Codd Normal Form) is a stricter form of 3NF. A relation is in BCNF if for every functional dependency $X \rightarrow Y$, X is a superkey.

- 3NF allows some non-superkey dependencies if the dependent attribute is a prime attribute, while BCNF does not.

⑧ Explain multivalued dependency with an example.

→ A multivalued dependency exists when one attribute determines multiple independent attributes in a table.

Example:

If $\text{student} \twoheadrightarrow \text{Hobby}$ and $\text{student} \twoheadrightarrow \text{Subjects}$, hobbies and subjects are independent but both depend on student.

⑨ What is 4NF? When is it needed?

→ A relation is in 4NF if it is in BCNF and has no non-trivial multivalued dependencies.

- If a table stores multiple independent multi-valued facts (like hobbies and subjects of students), we decompose it to remove redundancy.

(10) Define Join dependency and 4NF?

→ A relation has a join dependency if it can be split into two or more tables and perfectly reconstructed by joining them, without losing or creating extra data.

4NF (Project - Join NF): A relation is in 4NF if every join dependency is implied by candidate keys. It removes redundancy from complex joins.

(11) What is denormalization? When is it useful?

→ Denormalization is the opposite of normalization. It intentionally introduces redundancy by combining tables to improve query performance.

Query speed is more important than storage, e.g. in data warehouse or reporting systems, then it is useful.

(12) Compare normalization and denormalization.

→ Normalization: Organizes data into multiple related tables to reduce redundancy and anomalies. It's best for transactional systems. (OLTP)

Denormalization: Combines tables to improve query performance, at the cost of redundancy. It's often used in analytics or reporting. (OLAP)

(13) What is decomposition? List types of decomposition.

→ Decomposition is the process of splitting a relation into two or more smaller relations to eliminate redundancy and anomalies.

Types:

a) Lossless-Join Decomposition: No data is lost when relations are joined back.

b) Dependency-Preserving Decomposition: Ensures all functional dependencies are maintained.

(14) Explain lossless-join decomposition with examples.

→ A decomposition is lossless if the original relation can be exactly reconstructed by joining the decomposed tables.

Example: Student (Roll No, Name, Dept) → S1 (Roll No, Name) and S2 (Roll No, Dept). Joining S1 and S2 on Roll No gives back the original student relation.

(15) What is dependency preservation?

→ Dependency preservation means that after decomposition, all functional dependencies of the original relation are still enforceable in the decomposed tables, without needing to recompute joins.

⑩ write the steps to normalize a relation to 3NF.

→ The steps are:

- i) make sure the table is in 1NF (no repeating groups, atomic values).
- ii) Remove partial dependency → 2NF.
- iii) Remove transitive dependencies → 3NF.

⑪ Explain attribute closure with an example.

→ The closure of a set of attributes (X^+) is the set of all attributes that can be functionally determined from X .

Example: FDs = $\{A \rightarrow B, B \rightarrow C\}$,
closure of $A = \{A, B, C\}$

⑫ What is a canonical cover?

→ A canonical cover is a simplified, minimal set of functional dependencies that is equivalent to the original set.

It removes redundancy and extra attributes.

⑬ List Armstrong's axioms.

→ Rules to infer all possible FDs:

- i) Reflexivity: if $X \subseteq Y$ then $X \rightarrow Y$.
- ii) Augmentation: if $X \rightarrow Y$, then $XZ \rightarrow YZ$.
- iii) Transitivity: if $X \rightarrow Y$ and $Y \rightarrow Z$, then $X \rightarrow Z$.

②0 Explain the use of Armstrong's axioms in DBMS.

→ They are used to:

- i) Derive all functional dependencies.
- ii) Compute closures of attribute sets.
- iii) Check equivalence of FD sets.
- iv) Find ~~extra~~ canonical / minimal cover.

②1 What is meant by minimal cover of FDs?

→ A minimal cover is a reduced set of functional dependencies that has the same meaning as the original but with:

- Only one attribute on RHS.
- No redundant FDs.
- No extraneous attributes on LHS.

②2 Explain the concept of schema refinement.

→ Schema refinement is the process of improving a database schema by applying normalization and decomposition. Its goal is to eliminate redundancy, preserve dependencies, and prevent anomalies.

Q3) What are anomalies in database design?

→ Anomalies are problems caused by poor schema design with redundant data. The main types are

- insertion anomaly
- update anomaly
- deletion anomaly

Q4) Define insertion, update and deletion anomalies.

→ Insertion anomaly:

- can't add certain data without unrelated data.
- Ex: Can't add a new course unless a student enrolls in it.

Update anomaly:

- Same data stored in many places; updating one requires updating all.

Eg: Dept Name change requires edits in every student row.

Deletion anomaly:

- Deleting one fact unintentionally deletes other useful info.

Ex: Removing the last student of a dept also removes dept info.

25) What is a transaction in DBMS?

→ A transaction is a single logical unit of work in a database. It may consist of one or more operations (like INSERT, UPDATE, DELETE, SELECT) that must either all succeed or all fail.

Example: Transferring money between accounts → deduct from A, add to B. Both steps together is one transaction.

26) List the properties of a transaction (ACID).

→ The properties are as follows:

i) Atomicity

- Either all operations of a transaction happen or none. ("all-or-nothing")

ii) Consistency

- A transaction takes the database from one valid state to another, maintaining rules / constraints.

iii) Isolation

- concurrent transactions should not interfere with each other.

iv) Durability

- once committed, changes are permanent, even if the system crashes.

②7 Define concurrency control and its need.

→ Concurrency control is the process of managing simultaneous transactions without conflicts.

Need:

- Prevents problems like lost updates, dirty reads, inconsistent data.
- Ensures correctness + maintains isolation in multi-user systems.

②8 What is a schedule? Define serial and serializable schedule.

→ The order in which operations of multiple transactions are executed are called schedules.

Serial schedule

- Transactions are executed one after another, no interleaving. Always correct but slow.

Serializable schedule

- Non-serial schedule that gives the same final result as some serial schedule. It allows interleaving but ensures correctness.

(29) Explain conflict serializability with example.

→ A schedule is conflict serializable if it can be transformed into a serial schedule by swapping non-conflicting operations.

Conflicting operations: Two operations are conflicting if:

- They belong to different transactions.
- They access the same data items.
- At least one is a write operation.

Example:

(T₁)

(T₂)

R(A)

R(A)

W(A)

W(A)

These conflict because both access A and one writes. It can be reordered non-conflicting operations to make it equivalent to a serial schedule, then it's conflict-serializable.

Sam
08/09/21